SCOUT Final Report
Team 1

DGIN 5201
Professors FE Bordeleau & Dr. Vlado Keselj
Dalhousie University
April 2026

Team Members:
Emily O'Grady (B00647905)
Patrick Abankwa (B01008392)
Bibekta Malla (B01071266)
Sydney Parsons (B01068132)

Abstract

SCOUT is a crowdsourced community-based web app that was developed to fill a current gap in the outdoor condition reporting market. We conducted initial user feedback surveys which confirmed our observation that there was a gap in the market for outdoor condition reporting. We developed a prototype of SCOUT that is hosted as a website (using Vercel) with features including an interactive map where users can browse current conditions or add new reports, detailed condition report form where users can submit images of conditions, location submission form where users can add new locations to the SCOUT database, user activity badges to encourage user engagement and activity. If there were further development opportunities, we would add additional features including favourite outdoor locations, enabling users to receive email updates on locations, and also build our B2B partnerships enabling monetization of the platform. Following initial prototype development, our user experience testing yielded positive results and constructive feedback for improvement. As the application grows, we plan to address the feedback from our user experience testing.

Table of Contents

Abstract 2
Introduction 4
Value Proposition 4
What We Originally Envisioned 5
What Does The Website Do 6
What Went Well and What Went Poorly 9
Milestones 10
User Testing 12
Architecture 17
- Overall Architecture of the Implementation 18
- Request and data flow 18
- Server and Client Responsibilities 18
- Architecture Diagram 19
- Frontend Technologies and Implementation Choices 19
- Backend, Data, and Integration Architecture 20
- Mapping and Geospatial Experience 28
- Testing, Deployment, and Maintainability 29
- Evaluation of Technology Choices: Advantages and Disadvantages 30
- Conclusion 31

The Future - If you had more time to work on the project? 31

Introduction

For the final project our goal as a team was to find a product that we could all use and benefit from. Through our initial team discussions, we identified our shared love of outdoor spaces. Putting this concept into an app has been a rewarding challenge as we create a product not just for the classroom but one we hope to continue after the class is over. Our app, SCOUT, is a community led application based on sharing real-time updates with your community to be social, bring awareness to hazardous outdoor conditions and community emergencies, share your favourite outdoor locations and get updates on your destination before you and your family even leave the house. With our primary audience being Gen Z and Millennials, we aspired to create a unique and engaging narrative for our users. As the user you are a “SCOUT”, you report back on the outdoor spaces you visit and earn badges for your activities and contributions to the community.

The current version of our prototype includes features to add trail reports to current listings, add new locations, collect badges, and explore the conditions of all the outdoor spaces our community has shared. We have future plans for our app as well but to meet the current need for outdoor community updates our SCOUT prototype has achieved and met the initial milestones we had set out to accomplish for our users.

There are currently apps that provide trail updates and community driven reports which would be direct competitors for SCOUT; these include AllTrails and GAIA GPS. AllTrails is a web and mobile application that offers account holders trail information to their online community using a freemium model (AllTrails, 2026). The content on this application is solely related to trails, and it is not always current. Overall, AllTrails operates as more of a social platform for users posting about their time on these trails. GAIA GPS operates similarly to AllTrails in that it has a freemium model. However, this app is not a social or community platform; it is more tailored to individuals looking to explore with a GPS (GAIA GPS, 2026). SCOUT fills the gaps in the competition because when building the app, we went right to the users and sought their feedback on what they wanted and needed. And as a team who are passionate about community and outdoor activity, we had firsthand experience of existing gaps in the market.

Value Proposition

SCOUT's MVP delivers one core promise to its users: Real-time, crowdsourced trail and outdoor condition updates built by Nova Scotia's outdoor community, for Nova Scotia's outdoor community.

Where current technology offers stale and often irrelevant information, SCOUT addresses an obvious need for up-to-date trail conditions that provide insight into the terrain before the user even leaves their home. SCOUT users can browse live trail condition updates provided by other hikers and submit their own 30-second updates in return.

The MVP is deliberately limited in scope and does not try to map all trails, incorporate API weather feeds, or serve all outdoor enthusiasts. Instead, its primary purpose is to do one task incredibly well: provide reliable and accurate trail conditions immediately in order to allow users to make an informed decision about whether or not to go.

At the heart of SCOUT is the basic premise of letting users know what trail conditions are like at the moment, as reported by a hiker who recently completed the journey.

What We Originally Envisioned

The initial concept for SCOUT was broader in scope. This is an ambitious start that was aimed at building a comprehensive platform with trail intelligence involving crowd reporting for conditions, maps with accessibility and parking layers, environmental warnings for wildlife, algae, insects, and more, as well as a social leaderboard and badge system. In addition, from the very beginning, the idea was to create business-to-business relationships for companies as a means of revenue generation and expand to cover multiple environments, such as trails, beaches, ski hills, and waterways concurrently.

The original value proposition was expansive: SCOUT as the single source of truth for all outdoor conditions in Nova Scotia.

User discovery data confirmed the core problem but also revealed where to focus. The MVP narrowed in three important ways.

Three specific findings drove this narrowing:

1. Trail surface conditions dominated the open-ended responses as the preferred metric for users.

7 out of 10 users spontaneously cited mud levels, ice, snow depth, and wet conditions as the hardest real-time information to find. The signal was clear enough to anchor the MVP around this single use case rather than spreading development across multiple environment types.

2. Pre-trip habit intent was exceptionally high.

90% of respondents said they would probably check SCOUT before leaving the house if a same-morning update was available. This validated the core habit loop and confirmed that the product does not need to be a full platform to earn a daily check-in. A focused, reliable condition feed is sufficient.

3. The badge and leaderboard system ranked last as a motivator.

The original vision leaned on gamification, digital badges, and leaderboards as the primary mechanism for driving user contributions. Discovery data ranked this last (Avg.

score 3.11 out of 5), behind community contribution (2.20 out of 5) and local discounts (1.70 out of 5). This redirected the contribution incentive strategy away from social status features toward the B2B reward partnership model, which could underpin both monetization and user motivation simultaneously.

The MVP value proposition has remained true to the original vision for the most part, aside from its approach to scope and how it drives user engagement.

Originally, SCOUT intended to build a platform. However, the delivered MVP is simply a tool. This does not mean that it is lacking in ambition, only that it is directly responding to what the users need based on their feedback. SCOUT's delivered MVP focuses only on one pain point (trail conditions), one habit (checking ahead of time), and is driven by discovery.

These are the correct elements to have in place to move towards building upon them with other functions like maps, beaches, ski hills, and hazard notifications. Once users trust the site for its ability to provide information about the trails, it will be much easier to introduce other features to their routine.

What Does The Website Do

Our website, which is temporarily hosted at https://scout-nine-jade.vercel.app/ is a crowd-sourced outdoor conditions app which presents users with near real-time data and information on the conditions of their favourite outdoor locations. The website provides a platform for individual ‘scouts’ to report on the status of a particular park, trail or outdoor space. These conditions are compiled into a map where users can filter and search for locations based on preferences such as ‘off-leash’ or ‘kid-friendly’. SCOUTs who report conditions can work towards earning different badges based on their contributions to the community. In a further iteration of our website, we would expand our business partnerships to include ski hills, golf courses and other outdoor locations which could use our platform to provide real-time data and attract customers. This would be a monetization strategy, as business partners would pay for search optimization and offer scouts rewards and discounts at their locations.

Arriving at the SCOUT website, a user can see immediately a feed of condition reports made by other recent users (figure 1). These condition reports include photos, text description of the location, condition report text, and tags which enhance search capabilities. The condition tags include ‘muddy’, ‘icy’, ‘off-leash’, and ‘kid-friendly’ and serve to simplify the condition reporting flow for users who would like to give quick feedback on a location. These condition reports are stored in a database with spatial information about the location, time of the condition report, and the reporting user.

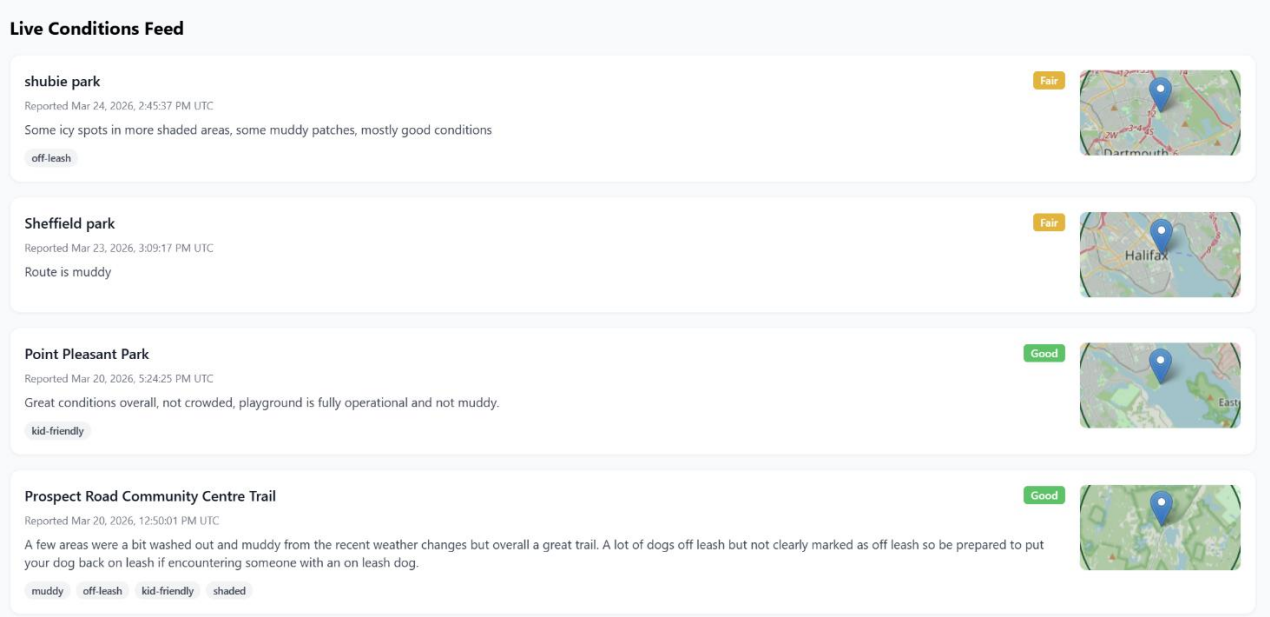

Figure 1 Live conditions feed on SCOUT, screenshot of production April 3, 2026

The next section of the scrollable SCOUT homepage provides users with the same database of condition reports but presented on a map view. This map shows pins of all the locations with condition reports. When a user clicks on a pin, they reveal the most recent report on that particular location. The map also includes a search bar and interactive condition tags (‘off leash’ and ‘kid friendly’) which can be used to filter the pins visible on the map (figure 2).

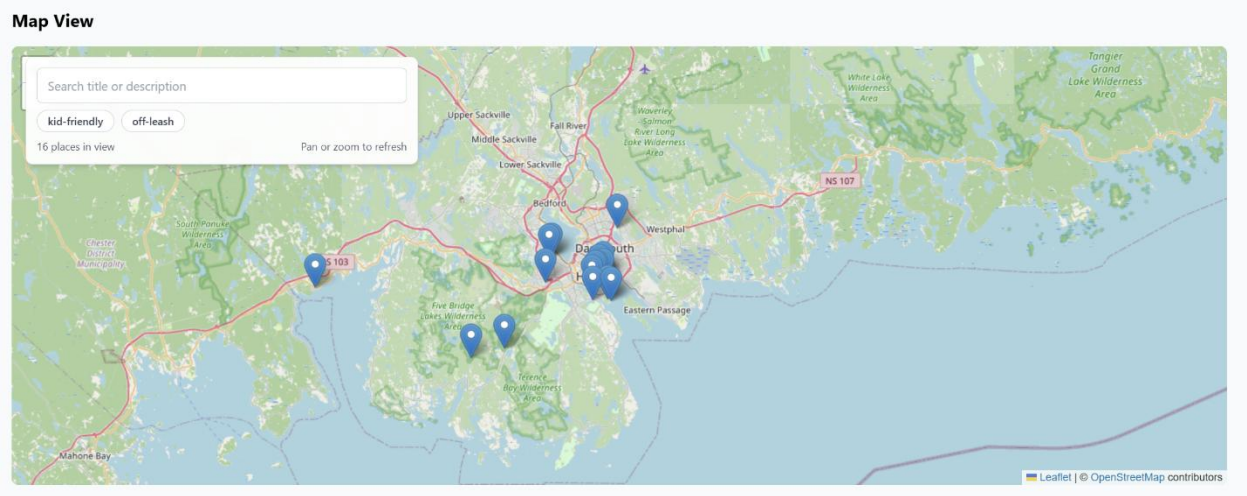

Figure 2 Map View section of SCOUT homepage, screenshot of production April 3, 2026

The homepage also includes a dedicated section for reporting a new condition and adding a location to the map. This brings a user to a fillable page where you can report details about a new location, interactively set the location with a map pin and then report on the condition of the location, including quick tags like muddy or icy and an overall star rating (figure 3).

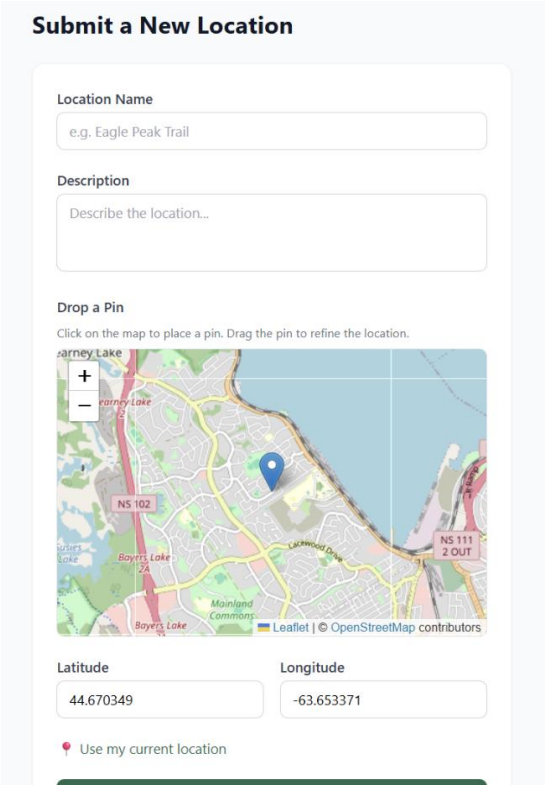

Figure 3 SCOUT form to submit a new location, screenshot from production on April 3, 2026.

The website requires users to log in before submitting a condition report and uses Google sign-in credentials to prevent users from experiencing new account fatigue (Beyond Identity, 2022), increasing ease of use. Once users have logged in, they can submit condition reports and view their own profiles which include all the condition reports they have made as well as their progress towards badges earned (figure 4). Our badge system includes different rewards and digital profile badges as users contribute more to the community, for example contributing a certain number of condition reports.

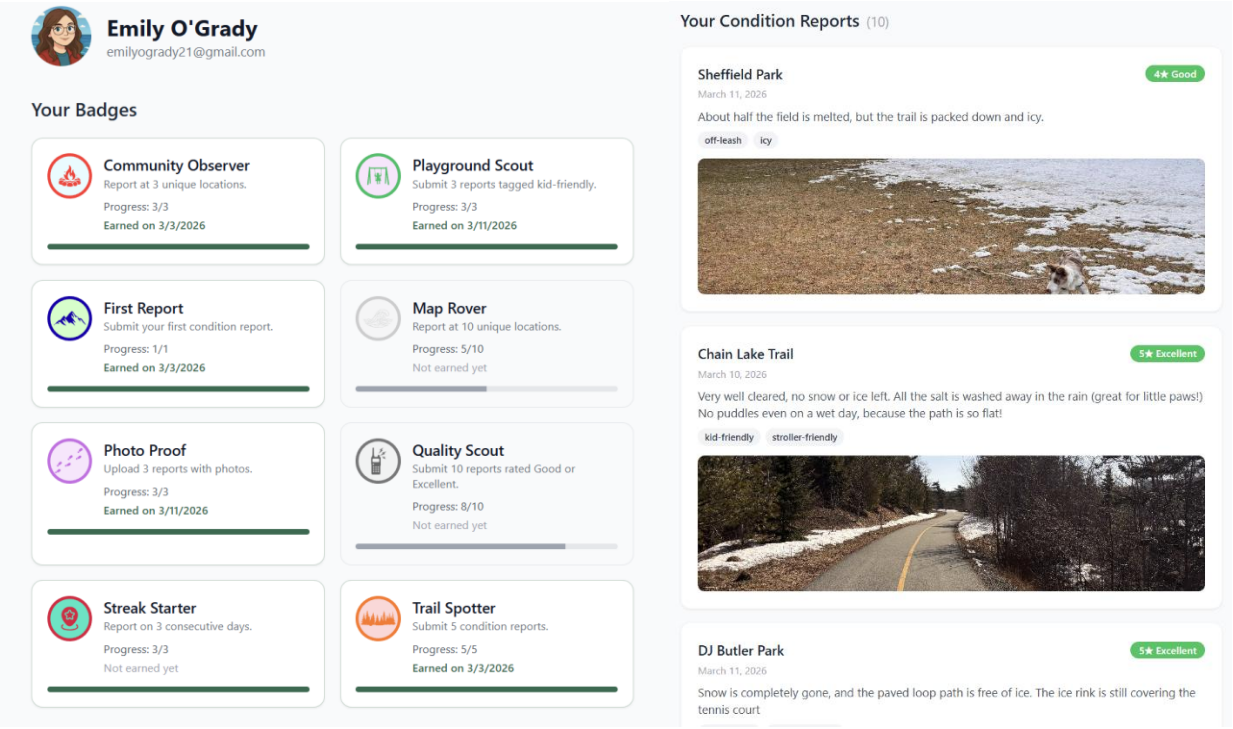

Figure 3 Screenshots from April 3, 2026, of SCOUT profile including badge progress (left) and historical condition reports (right)

What Went Well and What Went Poorly

The communication within our team has been very consistent, which made planning and dividing roles very easy. The team members offered to help and support other members where they were able to, which was a great success in our team dynamic.

Overall, the project went very well. Our team worked closely on idea generation, planning, and design, and it was only as we diverted into development and user testing that our roles became more separate. And even as we split up the roles, each of us worked diligently on our parts.

Our biggest hurdle was setting up our GitHub repository for our team to collaborate on. Since most of our team were new to this workflow of development, it was the first time we were connecting to a GitHub repository to collaborate with others. In the process of setting up node.js and npm which manage the packages and dependencies in the repository, there were some issues as we were setting them up on different OS (mac vs windows). Through collaborative problem solving and online forums we were able to fix these issues and start our collaboration on SCOUT.

As we started working on different sections of development, we took note of certain areas of overlap. Following this we needed to work out a way for each of us to contribute but also not work on parts that our teammates had done. After we built the initial site and started more specific design work, we did some trial and error on how we were going to divide tasks. We ultimately decided we split up sections of the code in our meetings so everyone's role was explicitly organized, we would make changes in our own individual dev branches and come back to our meeting the following week with our individual deliverables completed. To ensure we didn't have any merge conflicts when our dev branches were pulled to main, Emily took the lead on version control and repository administration, ensuring that all updates were synced to main on both GitHub and GitLab.

Milestones

Before the Early Specification presentation, the team built a strong foundation from the very beginning of the project. Starting January 21, we mapped each member's skills and interests, which made work allocation very easy. Through our initial meetings, we decided on our idea of SCOUT as a crowdsourced, real-time outdoor conditions platform. We gave it a clear identity, finalized name, logo, branding concept with SCOUT badges, and chose green as the brand color. By February, we were able to finalize the value proposition, select UI/UX design, and have an initial plan draft for user testing. This groundwork before Early Specification is what made the three main milestones achievable.

During the Early Specification presentation on February 26th, the team presented three key milestones, which were:

Milestone 1: Early Prototype (by March 10)- To build a basic website structure and set up database schema.

Milestone 2: User Feedback (by the end of March) - Have a working demo with at least two features and three sample locations to test and get feedback.

Milestone 3: Final Demo (by start of April)- An MVP version with at least four features and a database with twelve real locations.

Milestone 1: Early Prototype by March 10 (Achieved)

We completed our working prototype before the deadline around March 3, as the core setup was already done. One team member impressively had already set up the complete technical infrastructure, hosting with Vercel, a PostgreSQL database through Neon, and Google OAuth for login. That early infrastructure setup was one of the

biggest reasons, this milestone was met on time. It gave everyone a stable environment to build without delays or bottlenecks.

After the March 6 meeting, work was divided clearly among backend and database, design and front end and user testing and the business side. Everyone worked in parallel. The original plan was to bring our Figma designs into Framer and use it to convert the design directly into code. However, it did not work as expected, so the front end had to be built directly from the code. We had some HTML and CSS experience, so building on that skill we used TypeScript in VS Code and took help from Claude AI to generate snippets and match the design flow. Two people were working on the front end simultaneously, both shared a love for design and logic, which meant version control became critical, and GitHub conflicts were a real and recurring challenge. Every merge had to be careful. There were a lot of trial and error, a steep learning curve for front end development specifically, and moments where things broke in ways that took time to diagnose. The back end was handled professionally and confidently. Despite all the hiccups during frontend we managed to show our prototype thanks to backend and technical infrastructure set up done way ahead of time. Without it, we might have failed to achieve this milestone. The Early Prototype presentation on March 10 also gave us a confidence boost and was well received.

Milestone 2: User Feedback by End of March (Achieved Successfully)

We delivered a demo site with two features live which were map view, condition reporting and a database populated with 5 to 6 locations. We completed one internal user testing and two external user surveys with a total of 20 participants. Around March 9, we were able to send out our survey, and 90% of users said they would check SCOUT before leaving the house if a real-time update was available. User responses from the survey easily validated the core problem we were trying to solve. Then around March 18, before sending SCOUT out to real users, we did an internal testing round with the team and a few close people. We spotted minor bugs and solved them as well as tidied up our platform.

We conducted user testing slightly later than we originally imagined because we were still refining the front end before putting it in the live for real users. Once we were satisfied, we moved into outreach with our friends and relatives. We were still on time; the milestone was achieved because user testing was planned in parallel with development of the prototype from early on. We ran two phases of user survey and feedback on a tight timeline with Google Forms and kept participation low friction which gave us clean and structured data.

Milestone 3: Final Demo by Start of April (Achieved)

For Milestone 3, we had already met the core targets. The two things we committed to in the Early Specification were 4+ features and 12 real locations in the database. We had already cleared both of those in Milestone 2. The map view, condition update posting, tag-based map view search, Google OAuth authentication, SCOUT badges and scrollable location card pages were all live and functional by the start of April. Almost every check-in with the professors was done, and this milestone was achieved.

Though we hit this milestone, usability testing revealed some genuine gaps. Users found the search feature difficult to navigate, the tag system caused confusion, and the badge system was functioning well but wasn't in user's best interest. Due to which, we have figured out a way to position and communicate the badges more clearly and differently, which will be discussed in the "what would we do if we had time" section of this report. These were not failures, rather guidance to improve. Most of our features function well, but we did figure out what needed more of our attention; all the small details matter, and we thrive to meet every user's expectation. This is the reason why we will be keeping all the feedback we got from users as our top priority in our backlog.

User Testing

This is a report on SCOUT's user testing program, which has two stages. The first is the problem interview test, consisting of 10 participants through Google Forms, to verify whether users experience the pain points SCOUT was created to solve, as well as users' intention to contribute. The second is the usability test, consisting of 10 participants for the SCOUT interface, with three key tasks. The results show that users indeed experience the pain points SCOUT was created to solve, and the key contribution mechanism is working well. Discovery/search needs to be addressed before SCOUT is released.

Original Vision & Hypotheses

SCOUT was born out of the notion that every hiker, beach enthusiast, and trail runner in Nova Scotia faces the information gap between what the trail guide tells you and what the trail really looks like. Although other tools and resources provide information on the conditions of the trails, SCOUT hypothesized that the outdoor enthusiast community in Nova Scotia would be able to provide its own crowdsourced feed of conditions.

Pain Hypothesis: Outdoor enthusiasts in Nova Scotia cannot get reliable information about the conditions of the trails. This leads to wasted time and frustration.

Gain Hypothesis: If users were able to get the conditions the same morning before leaving the house, they would be able to get into the habit of checking the conditions.

A two-phase approach was developed for testing both hypotheses. Phase 1, User Discovery was designed to validate the existence and intensity of the pain. Phase 2, Prototype Usability Testing was designed to validate the existence of the gain and the effectiveness of the design in helping users experience the gain.

Testing Protocol

The program used a two-phase approach for testing, which included validating the problem and evaluating the interface.

Phase 1 – User Discovery Interviews

Conducted from March 9th to 10th, 2026, for 10 participants using Google Forms, Phase 1 aimed at validating the pain point, measuring the intent for a pre-trip check, and determining the incentives for contribution. It used an online survey for Gen Z and Millennial hikers in Nova Scotia.

The KPIs were set for a pre-trip check rate of ≥70%, contribution intent ≥40%, and high trust in live data.

Phase 2 - Prototype Usability Testing

This phase was conducted on March 18-23, 2026, using a Google Forms post-task survey on 10 participants. Phase 2 tested whether the participants were able to complete three main tasks on the SCOUT prototype without assistance. Participants used the SCOUT prototype independently and then completed a post-task survey. Observer notes recorded the real-time Section B score on a 0-3 scale.

The three main tasks to be completed by the participants were: T1 - Read trail conditions for the Bluff Trail; T2 - Post a mud warning update; and T3 - Find a dog-friendly beach with a 24-hour update.

The Section B scoring system was based on the following criteria: 3 (Expert) - Completion without delay and without hesitation; 2 (Satisfactory) - Completion with some friction; 1 (Needs Work) - Completion with a verbal hint; and 0 (Critical Fail) - Failure to complete the task. The goals were to attain a score of ≥2.4 and a success rate of ≥80%.

Phase 1 Results - User Discovery

The discovery survey had 10 respondents over the course of two days. The results were excellent validation of the pain hypothesis and the gain hypothesis.

Pre-Trip Check Rate: 90% - PASS. ≥70% target. 9 out of 10 users said they would check SCOUT before leaving.

Contribution Intent: 90% - PASS. ≥40% target. 9 out of 10 users said they would contribute.

Live Data Trust Score: 8.5/10 - PASS. 7 out of 10 users gave 9 or 10 for trust in live crowdsourced data vs. static data.

Pain Confirmed – Open Text Responses

When asked what information is hardest to find in real-time outdoors, the data points tightly around SCOUT's fundamental proposition:

- Trail surface conditions (mud, ice, snow, wetness) – 7 of 10
- Trail open/groomed status – 4 of 10
- Environmental hazards (insect activity, blue green algae) – 2 of 10
- Beach conditions (wind, water temp) – 1 of 10

The pain is confirmed. 70% of the users independently confirmed the same information needed: real-time trail surface conditions, without any options being presented. This is a powerful indicator for product direction.

Incentive Heatmap – What Motivates Contribution

Participants were asked to rate four incentive options from most to least motivating (1=most, 5=least). The results were directly relevant to MVP feature prioritization:

- Local gear/café discount – average rank 1.70 (85% most motivating)
- Helping the community – average rank 2.20 (70%)
- Leaderboard ranking – average rank 2.89 (45%)
- Digital SCOUT badge – average rank 3.11 (35%)

The badge and leaderboard feature, which was at the heart of our engagement strategy, was ranked last. Tangible rewards were ranked first, supporting our B2B partnership business model as an effective monetization and retention tool.

Phase 2 Results - Prototype Usability Testing

Ten participants completed three tasks with the SCOUT prototype in March 2026. Results show a product with high contribution potential but considerable room for improvement regarding usability in discovery and search.

- **T1 - Read Trail Condition (Bluff Trail)**: Average score 1.30/3 (≥2.4) | Success rate 50% (≥80%) - FAIL
- **T2 - Post Mud Condition Update:** Average score 4.30/5 (≥4) | Success rate 80% (≥80%) - PASS
- **T3 - Find Dog Friendly Beach (24hr Filter):** Partial result | Success rate 50% (≥80%) - WATCH

T1 - Trail Condition Readability - FAIL

This is the most critical finding. Three participants failed to find information regarding the trail condition altogether. Two participants were only partially successful. Specific issues experienced include:

- Results not shown for search bar entries 'Bluff' and 'Dog Friendly'.
- Too much screen real estate is dedicated to the search menu, obscuring the map view
- Legend for trail condition too small to comfortably view
- App misunderstanding intent to add a new trail where an existing trail exists

T2 — Posting a Condition Update (PASS)

The contribution flow is the strongest feature of the prototype. Not only did 50% of participants rate the contribution of a mud warning with a perfect score of 5 out of 5 for ease of use, but no participants scored it lower than a 3. One participant spontaneously commented that “it wasn’t difficult to update”. This is SCOUT's core gain creator working correctly.

T3 — Filtered Beach Discovery (WATCH)

50% of participants found the beach quickly, but 20% were unable to determine whether the result was updated within 24 hours. Participants were confused by the difference between 'dog-friendly' and 'off-leash' as separate filters.

Pain Relievers & Gain Creators — Evidence Assessment

Based on the results of both testing phases, the following assessment has been compiled to determine which parts of the app prototype deliver on the value proposition and which parts do not.

- **Real-time Trail Condition Feed - Pain Reliever: Confirmed.** 7 out of 10 Discovery users rated this as their #1 pain point. 90% said they would check SCOUT before heading out.
- **Crowdsourced Updates - Trust in Live Data - Pain Reliever**: **Confirmed**. 8.5 out of 10 average trusts in live vs. static data. 70% rated 9 or 10. This indicates users' trust in crowdsourced updates.
- **Trail Search & Map Navigation - Pain Reliever: Not Yet**. 3 out of 10 users were unable to locate the information they needed. Search does not return results. The design must be changed before release.
- **Contribution Flow - Posting an Update - Gain Creator: Confirmed.** 4.30 out of 5 average eases. 80% rated 4 or 5. No critical failures. The flow was easy and fast.
- **Filtered Search - Dog Friendly - 24hr - Gain Creator - Partial**. 50% success rate on T3. The 24-hour filter isn't visible enough. The taxonomy of the filters needs to be worked.
- **SCOUT Rewards - B2B Discounts - Gain Creator - Validated Concept.** #1 reward incentive - 1.70/5. Not implemented in the prototype. The next feature is to be built.

Reflection & Next Steps

User testing has reinforced the hypothesis and identified one key omission. The approach has provided validation of the product strategy while identifying the key pre-launch tasks.

What Worked Well

- By utilizing Google Forms for both phases, the ease of contribution was maintained, and clean data was collected efficiently.
- The open-ended Q1 in Phase 1 was the single most valuable piece of information collected. Allowing the user to self-describe the pain point removed the confirmation bias.
- A shared rubric in Section B provided the team with an objective system for assessing task completion rather than relying on subjective experience.

What We Would Do Differently

- Human in-person think-aloud sessions for usability testing. While the post-task survey was effective for measuring outcomes, we didn't have the opportunity to observe the confusion at the time.
- Test with a higher sample size (15-20 participants) for better confidence in calculating KPIs.
- Concurrently administer the B2B survey with Phase 1 testing to have all three data streams available at the same time for the decision window.

Priority Fixes Before Next Test Round

- Rethink the design of the trail search page, so 'find' is distinct from 'add'.
- Make the color-coded condition legend interactive.
- Make the 24-hour recency filter visible on beach and trail result cards.

Everything we see tells us to keep going. Users are leaning into the live data, and the core flow is already clicking for them. We've found an opportunity to clarify search filtering to enhance user experience.

Architecture

SCOUT is a community web platform, and the core purpose is to provide near real time outdoor condition information that is easily accessible to potential adventurers. The web framework needs to provide an interactive user interface which can support the reporting and retrieval of outdoor condition data. This framework was built with some careful considerations towards user accessibility, simplicity, and rapid prototyping development cycles. The website uses a modern full-stack web architecture that allows the product to be rapidly developed and easily extended. The main goals on the website include browsing location information geospatially, reading the most recent condition reports, submitting new locations, and updating conditions of existing locations with optional photos. Users are also able to view their personal contributions via their profile along with their earned badges.

This full-stack web architecture is built around Next.js 15, in TypeScript with Tailwind CSS, React components, auth.js for Google sign-ins, a PostgreSQL database on Neon with PostGIS, leaflet and OpenStreetMap for location display, Cloudinary for photo uploads and a Vercel hosting deployment. This implementation has built an end-to-end MVP which has allowed us to meet our target for robust feature implementation during rapid prototyping. This framework is also modern and flexible enough to be easily expanded during future development cycles. We used both GitHub and GitLab to take

full advantage of some of the integrated features of Vercel and GitHub, while meeting the GitLab submission requirements.

Overall Architecture of the Implementation

This web project is a full-stack application which operates as a monolithic repo, available on both GitHub and GitLab, with internal layers that add clarity to the structure. Next.js is used as both the frontend framework and backend runtime for simplicity of implementation. The codebase has an app layer for pages, layouts and API routes, a component layer for reusable UI and client-side interactive features. There is also a service layer for authorization, database access, Cloudinary uploads, condition reporting tag and badge logic. The database layer has schema and migration scripts, and shared types are available in a distinct layer as well.

Request and data flow

A typical ‘read’ flow, where a user is accessing data about an outdoor location, follows from a user opening the home page. The server component first fetches the locations and recent conditions from the database, then displays them on the client-side map. The user’s browsing is enhanced by dynamic filtering and viewport-based fetches. A typical ‘write’ flow, where a user is updating the condition of a location or adding a new location follows from a user logging in via Google Sign In. When a user submits a new location or condition report the request goes through a protected API route, the request is validated and rate limited. The data is stored in PostgreSQL. If an image is included, it is uploaded to Cloudinary and stored as a URL. The user profile flow is activated when a user views their profile. Their historical reports will be displayed, and their badge progress is computed from report history with persisted data to optimize efficiency.

Server and Client Responsibilities

The server side is responsible for rendering main page routes, authentication and session handling, direct database querying, and API route validation and writing. It also handles media uploads. The client side is responsible for interactive map behavior, geolocation access, pin placement on the submission page, form state management, and preview photos before uploading. Leaflet components are dynamically imported due to some issues with server rendering; the current form relies on browser APIs to ensure a clean experience and prevent errors.

Architecture Diagram

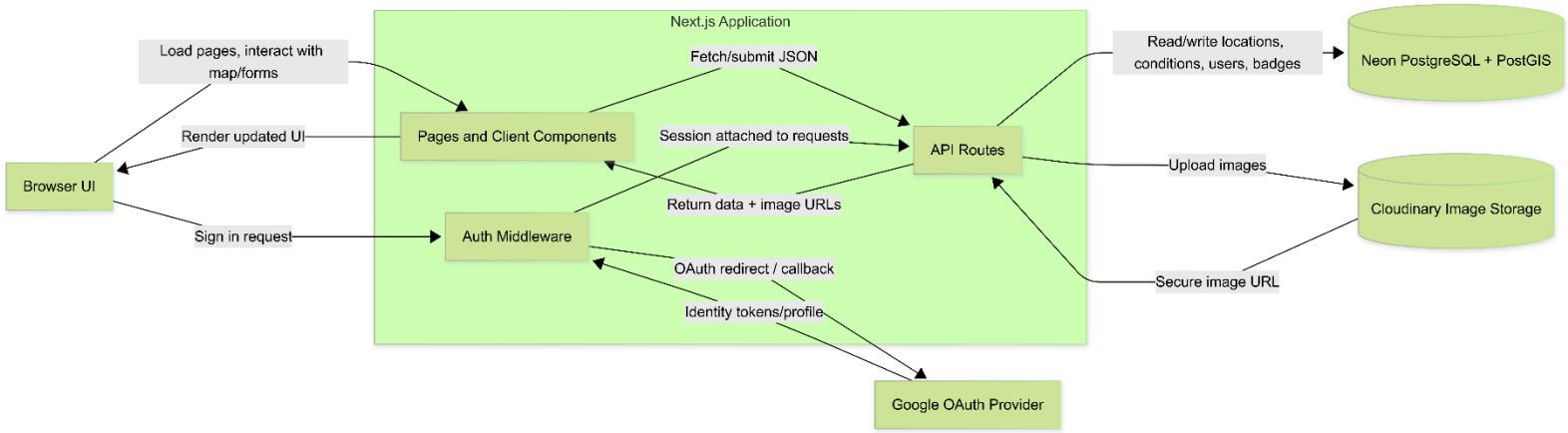

Figure 4 Full architecture diagram for SCOUT web application, made with mermaid.

Frontend Technologies and Implementation Choices

Next.js 15 and React

Next.js was chosen because it supports page routing and API routing in a single framework. The server components reduce the need for separate backend features, and it supports hybrid rendering and revalidation. This tech was also chosen based on existing expertise within the group and the desire to build with a modern and extensible stack that would offer maximum benefit of student experience for future projects. There are route-based pages for profile, submission, and location details.

Some advantages of using this framework include unified frontend and backend development, a clean and easy project structure for a small team, and simple deployment and integration with Vercel. There were some limitations such as the added complexity of server/client component boundaries, and some dynamic import workarounds with Leaflet, but overall, their effect on the team's workflow was minimal.

TypeScript

Typescript was the language of choice across the codebase, and allowed us to implement complex pages, API handlers, and shared models. Shared type interfaces were centralized in src/type/index.ts. This provided some clear benefits including better type safety across frontend and backend and reduced runtime bugs during refactoring. Some disadvantages are that it is more verbose than JavaScript and requires some extra type assetions for database results and third-party libraries. There was also a learning curve for the front-end team to translate from traditional HTML/CSS experience to TypeScript.

Tailwind CSS

Tailwind allows rapid development of styling and responsive layout with our custom color extensions and global overrides (src/app/globals.css). The fast iteration potential worked well with our development cycle while still providing consistent spacing and styling without the need for maintenance of large custom CSS files. Some potential disadvantages of this approach include the complexity of the class-heavy code, which can reduce readability.

UI and interaction design

The key elements of the front-end development included sidebar navigation on the home page, which was optimized for mobile with a hamburger style menu. The map block with overlaid filter and search was a major challenge and has been identified as an area for improvement in future rounds of development. The submission form with a map block to identify location was a success, built on iterative feedback from the team and internal testing. Condition reporting with photo preview required some advanced interactive techniques to ensure that photos loaded seamlessly and that front facing errors for users were informative and matched with the restrictions of the Cloudinary platform. The profile page with badge progress visualization is another opportunity for optimizing front-end design to encourage users

Backend, Data, and Integration Architecture

API route design

The backend endpoints are handled by Next.js with route handlers in src/app/api. The main endpoints are for authentication, location filtering, condition submissions, and photo uploads. The API layer performs validation, sanitization as well as rate limiting, and safe response shaping. Across the API surface, the design is consistent: write operations require authentication, user records are synchronized into the database before inserts, and abuse-prone routes are protected with in-memory rate limiting. The implementation lives in the App Router route files under /api, with most business logic delegated to shared helpers in auth.ts, /db.ts, rate-limit.ts, tags.ts, and cloudinary.ts.

Authentication is handled in api/auth/[...nextauth]. The authentication endpoint is intentionally thin. The route file at src/app/api/auth/[...nextauth]/route.ts simply re-exports the GET and POST handlers produced by NextAuth in auth.ts. That means the real behavior is defined centrally in the auth configuration rather than in the route file itself.

Scout currently uses Google OAuth as its authentication provider when the required environment variables are present. The auth helper supports both AUTH_GOOGLE_ID / AUTH_GOOGLE_SECRET and GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET, which makes local development and deployment configuration more flexible. If those credentials are missing, the app does not crash at import time but instead it logs a warning and starts without a configured Google provider. That is a useful resilience feature for build and preview environments.

The authentication flow also performs data synchronization with the application database. During sign-in, the callback in auth.ts:23 upserts the user into the users table, ensuring that the app has a persistent internal user record for every authenticated session. The session callback also copies the provider subject identifier into session.user.id, which is important because later write endpoints depend on that id for authorization and foreign-key relationships. The route additionally overrides the sign-in page so unauthenticated users are sent back to the home page rather than a separate auth screen.

The locations endpoint\ supports reading filtered location data with GET and creating new locations with POST.

For GET requests, the endpoint accepts several optional query parameters. Geographic filtering is supported through south, north, west, and east bounds. These values are validated as numeric coordinates before use, and invalid bounds produce a 400 response rather than reaching the database. The route also validates that south is not greater than north. In addition to bounding-box filtering, the endpoint supports repeated tag query parameters and a free-text q parameter. Tag validation is strict and only tags listed in tags.ts as MAP_FILTER_TAGS are accepted. At the moment those tags are limited to kid-friendly and off-leash, which means the API is intentionally conservative about what map filters can be used as it would be unhelpful for users to filter the map with tags like ‘muddy’.

After validation, the route delegates to getLocationsWithFilters in db.ts:30. That helper builds a SQL query that combines spatial bounds, keyword matching, and tag existence checks. One notable implementation detail is that it correctly handles antimeridian-crossing map bounds by detecting when west is greater than east and adjusting longitude logic accordingly. The query also aggregates related condition data, returning each location with condition_count and latest_rating. Before responding, the route strips the created_by field so internal ownership data is not exposed to the client.

For POST requests, the same route becomes a protected creation endpoint. It first requires a valid authenticated session via auth(), returning 401 if the user is not signed in. It then applies rate limiting using the locations namespace from rate-limit.ts. By

default, that limit is 10 requests per 10 minutes per authenticated user, though the values can be overridden by environment variables.

Before inserting the new location, the route upserts the user into the database again. This is a pragmatic safeguard for stale sessions or production edge cases where a session exists but the corresponding users row does not. The request body must include name, description, latitude, and longitude. Coordinates are parsed and range-checked, and invalid values return 400. Once validated, createLocation in db.ts:131 inserts the record and also generates a PostGIS geometry point using ST_SetSRID and ST_MakePoint. The API then removes created_by from the response and returns the created location with status 201.

The detail endpoint at src/app/api/locations/[id]/route.ts is read-only and returns a single location along with recent condition reports for that location. The route extracts the id from the dynamic path segment and uses two database helpers: getLocationById and getConditionsByLocationId from db.ts.

The endpoint first checks whether the requested location exists. If not, it returns a clean 404 Location not found response. If the location exists, the route fetches the most recent condition entries associated with it. The helper currently limits this to 20 condition records and orders them by condition_date descending, which keeps the payload focused on recent trail information rather than the full historical archive.

The response is deliberately sanitized. The route removes created_by from the location object and removes user_id from each condition object. At the same time, the underlying SQL query joins the users table and includes user_name and user_image, so the frontend can show attribution without exposing the raw internal user id. That is a useful privacy and UX compromise so the app can display who submitted a report, but it does not leak direct user identifiers in the API payload.

The condition creation endpoint is POST-only and exists to accept new trail-condition reports from authenticated users.

Like location creation, this route requires a valid session and uses the conditions namespace in the shared rate limiter. The default limit is 20 requests per 5 minutes per user, which is more permissive than location creation because condition updates are expected to happen more frequently. The route also performs the same upsertUser safeguard before writing, ensuring that the submitter exists in the users table before a condition row is inserted.

The required request fields are location_id, condition_date, rating, and description. The route validates rating as an integer from 1 through 5. It also supports two optional pieces of metadata: photo_url and tags. Tags are normalized to trimmed lowercase strings, deduplicated, and then validated against QUICK_TAGS from tags.ts. Current

accepted quick tags include kid-friendly, off-leash, muddy, icy, shaded, crowded, and stroller-friendly. If any submitted tag falls outside that whitelist, the route rejects the request with status 400.

After validation, createCondition in db.ts:154 inserts the report into the database. The API strips user_id from the returned object before sending the response, again avoiding unnecessary exposure of internal identifiers. The response status is 201 on success, and rate-limit headers are attached just like the other protected write routes.

The photo upload endpoint handles image uploads for condition reports. It is POST-only, requires authentication, and explicitly declares a Node.js runtime because it depends on Buffer handling and the Cloudinary SDK. That runtime choice matters in a Next.js App Router application because some routes can otherwise run in Edge contexts where those Node APIs are unavailable.

The request format is multipart form data with a file field named file. The route validates that a file is actually present and that the uploaded content type is one of JPEG, PNG, WebP, or GIF. It also enforces a 4 MB upload size limit before attempting the external upload. That protects the route from common serverless request-size failures and keeps costs and latency under control.

After validation, the file is converted from an ArrayBuffer into a Node Buffer and passed to uploadImage in cloudinary.ts. That helper lazily configures Cloudinary using environment variables and uploads the image into a scout folder, returning the secure_url from Cloudinary. The API then responds with a simple object containing the uploaded image URL.

Error handling in this route is notably more refined than a generic catch-all. The helper function maps common Cloudinary and payload-size failures into clearer user-facing error messages. For example, missing Cloudinary credentials are translated into an Image upload service is not configured on the server message, while oversize payloads become a 400 response with a clear 4 MB limit message. The route is also rate-limited, with a default of 10 uploads per minute per user.

All of these endpoints have some consistent design choices which helps the cohesion and maintainability of the codebase. Scout separates route handlers from business logic. The route files handle HTTP concerns such as parsing, validation, authentication, and response shaping, while the actual persistence logic lives in db.ts. That keeps the API layer relatively clean. Second, the API deliberately sanitizes responses. Sensitive internal fields such as created_by and user_id are removed before objects are returned to clients. This is a modest but important privacy and encapsulation measure. Third, the rate limiting strategy is centralized in rate-limit.ts. The current implementation uses an in-memory global Map keyed by route namespace plus user id or IP address. That is

straightforward and effective for a single runtime instance, though in a distributed deployment it would not provide globally shared enforcement across instances. That is worth noting as a scalability consideration. Fourth, the system relies on a small number of controlled enums for user-submitted metadata, especially tags. This reduces invalid input, makes the UI easier to drive from known values, and improves downstream filtering reliability.

Authentication with Auth.js v5 and Google OAuth

Scout uses Auth.js v5 with Google OAuth as its authentication solution rather than implementing a custom username-and-password system. The dependency is declared in package.json:23, and the authentication configuration is centralized in auth.ts:24.

In this project, Google is the only configured provider. The setup in auth.ts:10 conditionally registers Google OAuth when the required environment variables are present. It supports both the AUTH_GOOGLE_ID / AUTH_GOOGLE_SECRET naming convention and the GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET naming convention, which gives the app some flexibility across local and deployed environments.

The authentication flow is straightforward. A user begins by signing in with Google through the Auth.js flow configured in auth.ts:12. Once authentication succeeds, Auth.js creates a session and makes that session available to the rest of the application. The project extends that flow in two important ways. First, during the sign-in callback in auth.ts:29, the user's record is upserted into the database, ensuring that every authenticated user also exists in the internal users table. Second, the session and JWT callbacks in auth.ts:43 copy the provider subject into session.user.id, which allows later routes to identify the authenticated user consistently when writing locations, conditions, or uploads.

Session-aware route protection is handled in middleware rather than duplicated inside every page. The middleware in middleware.ts:4 wraps requests with the Auth.js auth helper, checks whether req.auth is present, and blocks access to protected paths when the user is not signed in. In the current implementation, the protected paths are /submit, /profile, and /api/conditions, as defined in middleware.ts:8 and matched in middleware.ts:19. For browser page requests, unauthenticated users are redirected to the home page; for protected API requests, the middleware returns a 401 Unauthorized JSON response instead. This creates a clear separation between public browsing and authenticated contribution features.

This approach has several advantages. The first is reduced implementation effort. By using Auth.js with Google OAuth, the project avoids building and maintaining password

storage, credential validation, account recovery, or low-level session management. The second advantage is user familiarity, Google sign-in is a common and trusted flow, which lowers friction for users compared with a custom registration process. A third advantage is architectural clarity. Because route protection is centralized in middleware.ts, access control is easier to reason about than if authorization checks were scattered across many pages and handlers. The sign-in callback's database upsert in auth.ts:29 is also practical because it keeps the authentication layer aligned with the application's own user records.

There are also limitations and tradeoffs. The most obvious is dependency on external OAuth configuration. The authentication flow depends on correct provider credentials and environment variables being present, so deployment errors in secrets management can break sign-in even when the application code itself is correct. Another limitation is the project's use of a beta release of Auth.js v5, shown in package.json:23. Beta software can introduce uncertainty through API changes, documentation gaps, or breaking updates between versions. Finally, authentication issues are often harder to debug in production than in local development. Problems involving callback URLs, trusted hosts, provider console settings, or environment secrets may only appear in deployed environments, which makes troubleshooting more involved than ordinary local application bugs.

Scout uses a relational PostgreSQL database, hosted through Neon, to store the application's core entities and the relationships between them. The schema is defined in schema.sql, and it models the system around five main tables: users, locations, conditions, badges, and user_badges. This is a strong fit for the project because Scout's data is highly structured and clearly relational. A user creates locations and submits condition reports, conditions belong to specific locations, and badge progress is tracked per user against a shared badge catalogue. That kind of interconnected data is better represented in a relational schema than in a loosely structured document store.

The users table stores the core identity information for each authenticated user, including an id, name, email, image, and created_at timestamp. This table acts as the parent entity for much of the rest of the system. The locations table stores each outdoor place submitted to the platform, including its name, description, latitude, longitude, geometry point, creator, and creation timestamp. Each location references the user who created it through the created_by foreign key. The conditions table stores trail and location reports submitted by users. Each condition is linked both to a location and to a user, and contains a condition date, numeric rating, text description, optional photo URL, tags array, and created_at timestamp. Together, these three tables represent the main community workflow: authenticated users create places and then attach condition updates to them over time.

The remaining two tables support the project's badge system. The badges table is a catalogue of all available achievement definitions, storing a badge id, name, description, icon path, metric type, and target threshold. The user_badges table is the join table that tracks each user's progress toward each badge. It stores the user id, badge id, current progress value, whether the badge has been earned, the time it was earned, and the last update time. Its composite primary key on user_id and badge_id ensures that each user has at most one progress record per badge. This is a good example of why a relational model is useful here so that badge tracking is naturally expressed as a many-to-many relationship with extra state attached to the link itself.

PostgreSQL is a good choice for this project because the application depends on data integrity and on relationships between entities. The schema uses primary keys, foreign keys, uniqueness constraints, and validation checks to enforce consistency directly in the database. For example, user email is unique, conditions cannot exist without both a valid user and a valid location, and ratings are constrained to a defined range. This gives the system stronger guarantees than relying only on frontend or API-level validation. PostgreSQL also provides mature query capabilities, which matter for a system that needs to join users, locations, and conditions efficiently and may later expand into richer filtering, analytics, and ranking queries.

PostGIS is an important extension in the design because Scout is fundamentally location-based. In the schema, each location stores latitude and longitude as regular numeric columns, but it also stores a geometry point column named geom. This matters because raw latitude and longitude are enough to display markers on a map, but a proper geometry column prepares the system for real geospatial querying. With PostGIS enabled, the application can later support features such as radius searches, distance-based sorting, proximity filtering, clustering, bounding-box optimization, or finding nearby locations from a user's current position. In other words, the project is not just storing map coordinates for display; it is laying the foundation for true spatial database operations.

The schema also includes several deliberate index choices to support performance. A GiST index is created on the locations geometry column so spatial lookups can scale efficiently as the number of stored places grows. Standard B-tree indexes are created on important foreign keys and date columns, including created_by on locations, location_id and user_id on conditions, and condition_date for time-ordered lookups. These are useful because many common operations involve retrieving all conditions for a location, all contributions by a user, or recent reports sorted by date. In addition, the conditions tags array has a GIN index, which is a strong choice for array membership queries. That makes tag-based filtering more efficient, especially as the platform grows and users rely more on structured labels such as muddy, icy, or kid-friendly.

The database design fits Scout's current needs and future direction well. PostgreSQL and PostGIS provide strong relational integrity, complex query support, and the spatial capabilities needed to grow from simple map pins into more advanced location-based features, while Neon's serverless PostgreSQL model aligns naturally with a modern Next.js cloud deployment workflow through environment-driven configuration and managed infrastructure. The main disadvantage is added complexity, since PostgreSQL with PostGIS requires more setup, schema management, and database knowledge than simpler options like SQLite or a document database, and serverless connection handling must be configured carefully to avoid issues with credentials, SSL, or deployment environments. Overall, the tradeoff is justified, because although this stack is more demanding to manage, it gives Scout a much stronger and more scalable foundation for structured data, geospatial queries, and future expansion.

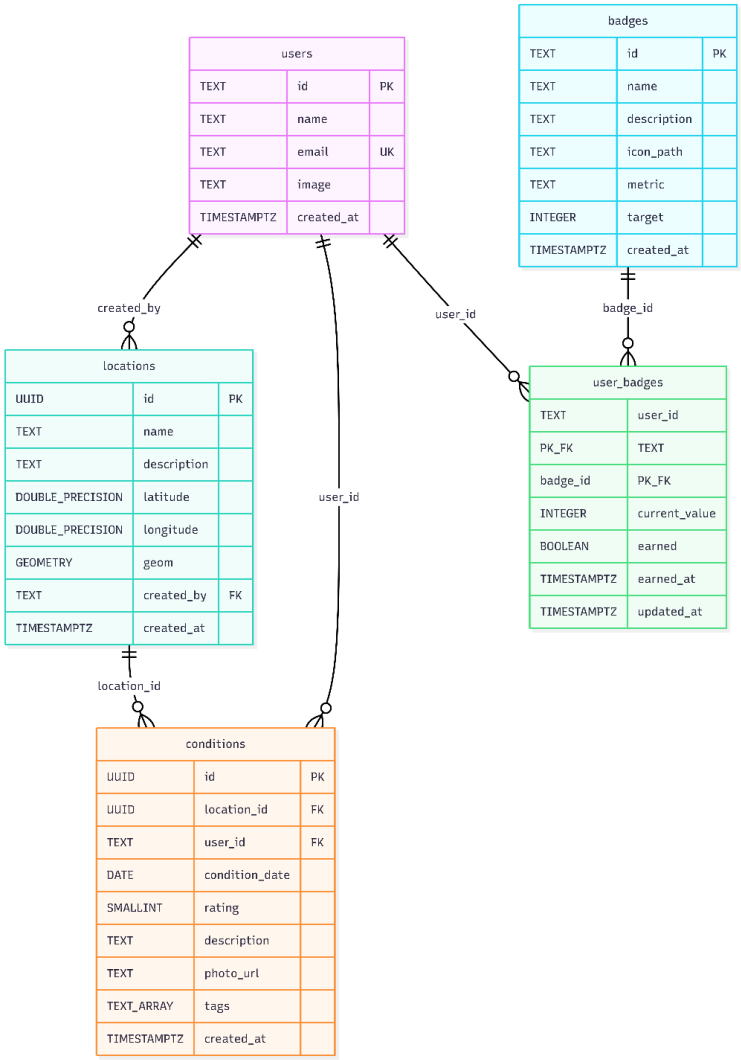

Figure 5 SCOUT database Entity Relationship Diagram showing all tables including users, user_badges, locations and conditions. Foreign key connections are shown by lines between tables. Made with Mermaid.

Data access strategy

Database access is centralized in src/lib/db.ts with helper functions like getLocationsWithFilters, getLocationById and upsertUser that streamline frequent database access techniques. The advantage of isolating the SQL logic from UI code is that it is more easily reusable, while being clear and transparent. The SQL logic is also more manual and requires broader database knowledge for maintenance and troubleshooting.

Cloudinary for image storage

We are using Cloudinary for photo upload and storage so that photos are not stored on SCOUT server space or directly in the database. The URL of the stored Cloudinary image is stored in the SCOUT database and associated condition report. This has the advantage of avoiding inefficient storage of large binary files and simpler image delivery and hosting which is a good fit for user-submitted media. The limitations encountered included the external dependency and configuration. The upload size limits quickly became an issue during internal testing so careful handling of user-facing errors was needed to give informative feedback and prevent user frustration.

Mapping and Geospatial Experience

Scout uses Leaflet together with react-leaflet to provide the interactive mapping features that sit at the center of the application. This can be seen across several parts of the codebase including the home page map which supports browsing all submitted locations, filtering results by visible map bounds and selected tags, and automatically recentering near the user's geolocation when permission is granted. The submit page uses a clickable, draggable pin map so users can choose precise coordinates for a new location; and the location detail page displays a focused map centred on one specific place. Rather than depending on a proprietary map platform, the project loads map tiles from OpenStreetMap, which is a practical choice for a student or community project because it avoids the cost and vendor lock-in of a paid SDK while remaining well supported by Leaflet. This approach has clear advantages, maps dramatically improve the user experience in a location-based application, and the Leaflet plus OpenStreetMap stack is flexible, mature, and cost-effective. However, it also introduces real engineering tradeoffs. Compared with commercial providers, OpenStreetMap-based solutions offer less integrated tooling and require extra work for performance tuning, custom styling, or advanced mapping features.

In addition, using Leaflet inside a Next.js application means dealing carefully with SSR and client-only concerns, since map rendering and browser APIs such as geolocation must be handled on the client side through dynamic imports and guarded runtime logic. Overall, the mapping stack is a strong fit for Scout, but it requires deliberate engineering decisions to balance user experience, cost, and technical complexity.

Testing, Deployment, and Maintainability

Testing approach

SCOUT relies on Vitest for unit testing. The unit testing would be a focus of future rounds of development and was not fully flushed out during this round. Some specific logic was tested due to bugs encountered during development and internal testing, such as the badge logic testing in src/lin/badges.test.ts. Ideally, business rules such as badge computation would be tested separately from the UI. Currently, the test coverage is mostly focused on utility logic and does not cover end-to-end flows or API integration.

Deployment and environment management

The project is configured for deployment with Vercel which was an extremely simple platform to learn and provided great functionality for rapid prototyping and development. Environment variables had to be input directly into Vercel for database connection, OAuth credentials, and Cloudinary connections. The deployment is very straightforward with automated set up and testing connected to the GitHub repository which automatically redeploys with every update and allows for visualization and testing of individual development branches which was very helpful during the rapid prototyping with team members. There were a few issues encountered with differences between local and deployment behavior which added a layer of complexity to troubleshooting but was overall manageable.

Maintainability observations

The clean and clear folder structure, with separation between UI, route handlers, and library helpers, shared reusable types and setup/migration scripts for the database, are all core strengths which enhance the maintainability and readability of the codebase. An increase in test suite would improve confidence and enhance maintainability, especially with multiple development team members. Some UI logic and styling could be decoupled in component files. In an operational setting, we may need to enhance the infrastructure to better handle a scaled userbase.

Version Control

Version control was key to facilitating group collaboration and encourage contributions from all group members. Connecting the codebase to GitHub and GitLab allowed us to take advantage of github action templates and Vercel integrations. This required setting up multiple origins to simultaneously push to GitHub and GitLab from the main development machine, while other developers were set up to push to individual development branches only on GitHub which streamlined pushes to main and protected the deployment environment from bugs.

Evaluation of Technology Choices: Advantages and Disadvantages

Choosing Next.js as the full-stack framework gave the team a single codebase for both the frontend and the API layer, which reduced coordination overhead and let a small team move quickly without splitting concerns across separate repositories or deployment pipelines. TypeScript reinforced this efficiency by providing consistent types across the entire project, from database query results in db.ts through to shared interfaces in types/index.ts and component props throughout the UI. Type errors were caught early rather than surfacing at runtime, and shared types meant that a change in the data model propagated predictably across the whole codebase.

PostgreSQL with PostGIS was a deliberate longer-term investment. While it added more setup complexity than a simpler embedded database, it ensured that location data was stored with proper geospatial support from the beginning, meaning the project is prepared for radius searches, proximity filtering, and spatial queries without requiring a future schema redesign. Pairing this with Neon's serverless hosting kept the deployment model modern and cloud-native.

Integrating external services such as Google OAuth through Auth.js and Cloudinary for image uploads dramatically reduced the scope of what the team had to build directly. Authentication, credential management, and media storage are genuinely complex problems; delegating them to established providers meant that development effort could stay focused on the application's core features. GitHub Copilot further accelerated development through AI-assisted code generation, helping the team produce boilerplate, implement validation logic, and explore patterns more quickly than writing everything from scratch.

These choices also introduced real complexity. Each external provider required its own environment variables, OAuth console configuration, and deployment secrets, which made local setup more involved and created more failure points in production. Next.js App Router's server and client rendering boundary added a recurring architectural concern, particularly around Leaflet and other browser-dependent libraries that required dynamic imports and client-only guards. Taken together, the project's deployment footprint, relying on Vercel, Neon, Cloudinary, and Google OAuth working in concert, is meaningfully more complicated than a self-contained local stack, and any one misconfigured service can break a user-facing feature entirely.

Conclusion

The Scout implementation follows a layered full-stack architecture in which the Next.js App Router handles both the frontend and the API surface, PostgreSQL with PostGIS manages structured and geospatial data, and external services provide authentication, media storage, and hosting. Every layer was selected in service of a clear design goal which was to deliver a location-based community experience where users can discover outdoor places on an interactive map, submit condition reports securely, and access live, community-driven information from any device. On balance, the chosen technologies achieved that goal effectively. The combination of Next.js, Auth.js, Leaflet, Cloudinary, and Neon enabled rapid development of a working, deployable product with strong core features and a coherent architecture. At the same time, each of those choices brought real integration complexity, from managing OAuth credentials and serverless database connections to navigating server and client rendering boundaries in Next.js. The result is a system that is capable and well-structured, but one whose operational setup is meaningfully more involved than a simpler stack would require. Taken together, the project demonstrates both the power and the tradeoffs of building a modern, cloud-native web application with a small team.

The Future - If you had more time to work on the project?

Under three months, SCOUT went from just an idea to a live working platform, but we were building it under a strict deadline. There were features, fixes, and ideas we had to leave behind to complete the MVP on time. This section covers everything in our backlog and what we would have done in our next iteration round.

Before that we iterated on two important things. The first one was merging the dog-friendly and off-leash filters into one as it created confusion among users during testing. They expected one filter to include another as they have the same meaning. Regardless of the timeline, we also felt there was a need to add a privacy policy, especially given that real-time location sharing can feel invasive without clear communication. Users should be able to know how their data is being handled. Everything below is what we will be working on next.

The first thing we would tackle is the search and discovery experience, because our usability testing made it clear where SCOUT falls short. Task 1, finding trail conditions for Bluff Trail had only a 50% success rate, well below our 80% target, and three users could not find the information at all. The root cause was that the live condition feed displays all trail updates chronologically across every location, so a user specifically looking for Bluff Trial must scroll through the entire feed to find it. Right now, we lack a separate “Search Trial Conditions” feature, which is why this is our number one priority.

Alongside this, the color-coded condition legend on the condition card needs to be interactive because some users in our test felt the need for a good description behind our labels such as “good” “fair” “excellent”. Though it is self-explanatory, we would be adding a tappable legend with written descriptions when hovered around the label. That would meaningfully improve the user's experience by making it easy for them to interpret what they are seeing. Task 3 showed that 20% of users could not confirm whether the result had been updated within 24 hours. So, the third fix is the timestamp as SCOUT's entire value is that the information is current, but if a user cannot see that immediately, then the value disappears. Though we have a timestamp of posted condition, having a timestamp badge which would say how many hours ago it was posted on every card would be easy for the user rather than having to calculate themselves. These are not big changes but based on our testing these are the ones which have the highest impact, before we jump into any new features.

Once those fixes are done, the next priority is location search from map view. SCOUT currently uses tag-based search, so you can filter by trail type such as "dog-friendly" but you cannot search by location. For a user standing in Dartmouth who wants a trail within ten minutes of their house, that is a significant gap. Letting users type a street name, city, or simply allowing SCOUT to use GPS coordinates, which we used in “add a new location” form, would solve this problem. We also have a feature that would add significant value, but we did not have time to complete as part of this round of development, which is the "saved favorites." Letting users bookmark trails and sharing them with their friends. It would likely be the most used feature for anyone who visits the same trail regularly, and it also fits naturally into how people plan trips together. Saved favorites also become the prerequisite for push notifications or email updates because

you cannot alert someone about their favorite places if you do not know what their favorite place is.

The badge system we built is something we are very proud of, but our Phase 1 incentive data told us that these digital badges ranked last as a motivator with an average rank of 3.11 out of 5. On the other hand, local gear and cafe discounts ranked first at 1.70 out of 5, so we would want to put the badge collection at the front, on your dashboard at login, which already has progress indicators, and connect those badges directly to tangible rewards. A SCOUT with enough badges unlocks a discount at a local outdoor shop. The badge becomes a loyalty point. That is the version of gamification that actually changes behavior, and it opens the door to the B2B partnership model we had planned. The idea is to give outdoor businesses like gear shops, golf courses, and ski resorts their own dedicated page on SCOUT where they can post live updates. For users, this means official updates sitting alongside community ones. From a business point of view, it is an excellent way to target users and brand visibility to the right audience. So, this would create a loop between gamification and monetization as a win-win situation for both users and businesses.

Additionally, there are several features that we would like to add to make SCOUT a valuable platform. The conditions feed currently runs chronologically, but we would introduce community-driven ranking where the most recent and most useful reports rise to the top, like the Reddit algorithm. Users could upvote an update that matches what they saw, and the feed would organize itself around quality. The more active the community, the smarter the feed becomes. We would also add in-app navigation so that tapping a trail and tapping "get directions" routes you to the trailhead.

The longer-term vision is a mobile app. SCOUT is only living in the browser right now, but an app is where SCOUT becomes a daily use tool. It would allow real GPS access without asking for browser permissions for every session; proper push notifications, and offline access would be given for trail sections with no cell signal. Alongside this, SCOUT launched a focused set of locations which was the right call for a prototype, but the platform's long term value scales directly with coverage. Phase 1 users identified hyper-local, time-sensitive conditions like trail surface, snow depth, water temperature and wind warnings as the hardest information to find anywhere. Expanding the database and making condition categories specifically tailored for each location type, so a beach has different fields than a ski resort and golf course, is how SCOUT becomes genuinely irreplaceable. The tag system would also grow to include options like stroller-friendly, wheelchair accessible, no cell service, requiring a parking pass, etc.

The order of this backlog is not arbitrary. It is driven by what our users showed us. The priorities at the top fix real failures we observed in testing. The priorities in the middle

build the habit loop that keeps users coming back. The priorities at the bottom expand SCOUT's reach and make the platform something people rely on.

References

AcknowledgeAI. (2024). *AcknowledgeAI.* https://aicontributionscale.my.canva.site/acknowledgeai

AllTrails. (2026). *Explore Trails.* AllTrails. https://www.alltrails.com/

Anthropic. (2026). *Claude Code*. https://claude.com/product/claude-code

Anthropic. (2026). *Claude* (claude-sonnet-4-6) [Large language model]. https://claude.ai

Beyond Identity. (2025, August 5). *Pros & cons of social logins*. https://www.beyondidentity.com/resource/pros-cons-of-social-logins

GAIA GPS. (2026). *GAIA GPS.* Outside Interactive Inc. https://www.gaiagps.com/

GAIA GPS. (2026). *GAIA GPS.* Outside Interactive Inc. https://www.gaiagps.com/map/

Government of Canada. (2023). *Privacy Notices.* Common Privacy Deliverables. https://www.canada.ca/en/government/system/digital-government/digital-privacy-playbook/privacy-notices.html

Mermaid. (2026). *Mermaid JS.* https://mermaid.ai/

Node.js. (n.d.). *An introduction to the npm package manager*. OpenJS Foundation. https://nodejs.org/en/learn/getting-started/an-introduction-to-the-npm-package-manager

Office of The Privacy Commissioner. (2024). *OPC Privacy Policy.* Privacy and transparency at the OPC. https://www.priv.gc.ca/en/privacy-and-transparency-at-the-opc/pp/#_why

Office of The Privacy Commissioner. (2025). *PIPEDA fair information principles.* The Personal Information Protection and Electronic Documents Act (PIPEDA). https://www.priv.gc.ca/en/privacy-topics/privacy-laws-in-canada/the-personal-information-protection-and-electronic-documents-act-pipeda/p_principle/

Perry, J. S. (2021). *Install Node.js, npm, and VS Code*. IBM Developer. https://developer.ibm.com/tutorials/learn-nodejs-installing-node-nvm-and-vscode/

Potter, R. (n.d.). *What is user experience?* Finalsite. https://www.finalsite.com/blog/p/~board/b/post/what-is-user-experience

Purdue OWL. (2026). *APA Formatting and Style Guide (7th Edition).* Purdue University. https://owl.purdue.edu/owl/research_and_citation/apa_style/apa_formatting_and_style_guide/reference_list_electronic_sources.html

SVG Repo. (n.d.). *Map vectors*. https://www.svgrepo.com/vectors/map/

The Basics Guide. (2024). *1% Rule — Participation Inequality.* The Basics Guide. https://thebasics.guide/1-percent-rule/

The Global Statistics. (2025). *Canada Social Media Statistics 2025 | Most Popular Platforms*. The Global Statistics. https://www.theglobalstatistics.com/canada-social-media-statistics/

Tomasis, R. (2023). *How to write an effective privacy policy for your website.* Wix Blog. https://www.wix.com/blog/how-to-write-website-privacy-policy

Trans Canada Trail. (2020). *Trail Use An Integral Part of Canadians' Lifestyles, Survey Finds.* https://tctrail.ca/news/trail-use-national-survey-utilisation-sentiers-sondage-national/

WebAIM. (n.d.). *Contrast checker*. https://webaim.org/resources/contrastchecker/

Appendix A – AI Badge

Our team members used AI throughout the development process. GitHub Copilot was an essential tool used to start the repository and build the framework for the website. Various LLM tools were used in the planning stages for brainstorming and creating a starting point for research in market validation and user testing. LLM tools were also used to create outline materials for this report to ensure that the sections met the expectations of the rubric and answered all the questions provided.

Appendix B – Questionnaires

User Product Research

Scout Product Research: Trail Condition Feedback and Relevance

Thanks for taking part in Scout's early research!

Scout is a trail condition app built for Nova Scotia's outdoor community, giving hikers, beachgoers, and adventurers real-time updates on conditions straight from people who were just there.

This short survey (5 minutes) helps us understand how you currently find trail and outdoor information and what would make you a Scout contributor. Your answers will directly shape what we build.

All responses are anonymous. There are no right or wrong answers — just your honest experience.

Q1. When planning an outdoor activity in Nova Scotia, what information is currently the hardest to find in real-time? (e.g., trail mud levels, beach wind conditions)

Long answer text

Q2. On a scale of 1–10, how much do you trust 'static' information from a week ago versus a crowdsourced update from 2 hours ago?

	1	2	3	4	5	6	7	8	9	10	
Trust static info equally	○	○	○	○	○	○	○	○	○	○	Much more trust in live updates

Q3. If you knew Scout had an update from this morning about your destination, would you make it a habit to check the app before leaving your house?

- ○ Yes, definitely
- ○ Probably
- ○ Maybe
- ○ Unlikely
- ○ No

Q4. What would motivate you most to take 30 seconds to post a condition update about a trail or beach? (Please rank your preferences, 1 being most motivating)

	1 (Most Motivating)	2	3	4
A digital 'Scout' ba...	○	○	○	○
A discount at a loc...	○	○	○	○
Simply helping the ...	○	○	○	○
A leaderboard rank...	○	○	○	○

User Testing Feedback

Scout Product Testing: Trail Condition Feedback, Usability, and B2B Validation

B I U

A comprehensive survey combining problem interviews with Gen Z/Millennial hikers, prototype usability feedback, and B2B validation for Scout, an outdoor condition crowdsourcing app.

Scout – Prototype Usability Test

Long answer text

Q1. Using the map, were you able to determine if the Bluff Trail is currently in good enough condition for a hike today?

- ○ Yes, easily
- ○ Yes, but it took a moment
- ○ Partially – I wasn't fully sure
- ○ No, I couldn't find the information

Q2. You noticed a trail is extremely muddy. How easy was it to update that status in the app to warn other Scouts?

	1	2	3	4	5	
Very difficult	○	○	○	○	○	Very easy

Q3 Follow-up: What made it difficult? (optional)

Short answer text

Q4. Were you able to find a dog-friendly beach nearby that had been updated by a user within the last 24 hours?

- ○ Yes, found it quickly
- ○ Yes, but it took a while
- ○ I found a beach but couldn't confirm the 24-hour update
- ○ No, I couldn't find one

Q5. Was there any point during these tasks where you felt you needed a 'Help' button or felt unsure of what to do next?

- ○ No, it was intuitive
- ○ Once or twice
- ○ Several times
- ○ Frequently

Q6 Follow-up: If yes, where did you feel stuck?

Long answer text

Appendix C – AI Chat logs

Some AI Chat logs have been provided as supplementary materials in distinct folders within the project repository. Some chat logs are available on GitHub. (https://github.com/EOGrady21/scout/agents?author=EOGrady21)

Initial Market Validation and name brainstorming:

https://claude.ai/share/98efa77a-781d-4fb3-85db-2080225ea70a

Scout User Testing Initial Scoping:

https://claude.ai/share/788bb3ee-cb43-40ac-809e-4f8c1d6792b5

Initial Terminal/Github setup: https://www.perplexity.ai/search/i-recently-pulled-commits-from-gI26gWCmQRStPP6npdoilQ?sm=d#30